

Architecting Agentic Software Systems

A Technical Portfolio and Professional Itinerary

Daniel Ray Edgar

daniel@nodebase.ca | github.com/dm3n | finsider.ai

June 26, 2026

Abstract

This document is a technical overview of my work designing, building, and deploying *agentic software systems* — autonomous and semi-autonomous programs that plan, act, and verify their own work under engineered constraints. I am the Chief Technology Officer of Finsider, where I built and deployed SYMPHONY, a supervised-autonomy software-delivery runner that drives product tickets to reviewed, evidence-backed pull requests. Before this I studied Computer Science at Queen’s University, then raised \$220,000 at a \$2.2M valuation as co-founder and CEO of an Antler Canada-backed company at the age of 19, and authored single-author research formalizing the reliability of multi-step language-model reasoning — validated against four frontier models to within roughly one percent of empirical accuracy. The unifying thread of this portfolio is a specific, defensible engineering thesis about autonomous AI, derived from that research and realized in production. This document doubles as a professional itinerary; my curriculum vitae is appended. Every system referenced is hyperlinked to its public source repository.

1 Introduction

I build artificial-intelligence systems for the highest-stakes work in finance, and I build the autonomous infrastructure that builds them. My work spans three layers that most engineers treat separately: the *theory* of why language-model systems fail, the *systems* that exploit that theory in production, and the *tooling* that lets a single engineer operate at the throughput of a team.

The defining specialization is *agentic software* — programs that decompose a goal into steps, take real actions in the world (writing code, querying corpora, operating a browser, transacting), and judge the quality of their

own output. Agentic systems are powerful and notoriously unreliable; the central engineering problem is making them trustworthy enough to point at production. The systems in this portfolio are my answers to that problem, and Section 3 gives the formal reason they are designed the way they are.

Selected facts. Chief Technology Officer of Finsider on a \$200,000 salary at the age of 20; raised \$220,000 at a \$2.2M valuation (Antler Canada Fund I SAFE) as co-founder/CEO before turning 20; single-author research validated against GPT-5, Claude Opus 4.6, Claude Sonnet 4.6, and Gemini 3.1 Pro; author of SYMPHONY, deployed live in a commercial engineering environment.

2 Core Technical Competencies

Agentic systems (primary specialization). Supervised-autonomy architectures; ticket-native state machines; acceptance-contract prompting; evidence-gated delivery pipelines; multi-agent orchestration with shared context; tool-use design; process supervision, watchdogs, and safe automated Git; failure recovery with bounded retry.

Applied AI / LLMs. LLM orchestration; Retrieval-Augmented Generation (Vertex AI RAG; corpus-per-workload; parallel multi-query retrieval with dedup/ranking); structured extraction with calibrated confidence; process reward models; frontier-model evaluation; Anthropic, Google Vertex AI, and Gemini 2.0 Flash APIs.

Languages. Python, TypeScript/JavaScript, Swift (iOS), SQL, Bash, L^AT_EX.

Full-stack. Next.js 16, React 19, Tailwind v4, shadcn/ui; Supabase (PostgreSQL, Row-Level Security, Auth, Storage), Firebase, Sanity (headless CMS); Stripe, Resend, Twilio; server-sent-event streaming, async job queues, web-hook pipelines.

Cloud & infrastructure. Google Cloud Platform (Compute Engine, Cloud Storage, Vertex AI), Vercel; self-hosted Proxmox cluster, Docker, `systemd`, macOS `launchd`, Tailscale; CI-aware multi-repository delivery.

Applied mathematics. Probability theory; Bernoulli and Hoeffding bounds; maximum-likelihood estimation; the delta method and confidence-interval construction; empirical model evaluation.

Financial domains. M&A Quality of Earnings; Canadian mortgage underwriting (GDS/TDS, OSFI guidelines, stress testing); financial-document AI; document-authenticity and fraud-signal detection.

3 Research Foundations: The Reliability of Agentic Reasoning

The design of every agentic system I build follows from a result I proved in *Uncertainty Propagation in Tree-Structured Language Model Reasoning* (github.com/dm3n/uncertainty-propagation). Model each step of an autonomous process as a Bernoulli trial with step-specific failure probability θ_i .

Theorem 1 (Chain success). *For an n -step reasoning chain $\mathcal{C}_n$ with conditionally independent steps,*

$$\mathbb{P}(\text{CORRECT} \mid \mathcal{C}_n, \boldsymbol{\theta}) = \prod_{i=1}^n (1 - \theta_i).$$

Corollary 1 (Exponential decay). *With mean failure rate $\bar{\theta}$, $\mathbb{P}(\text{CORRECT}) \leq (1 - \bar{\theta})^n \leq e^{-n\bar{\theta}}$.*

The practical consequence is severe: a long, unsupervised agent run decays exponentially in reliability, so it *cannot* be trusted to ship. The paper further proves that a k -ary, depth- d tree with majority aggregation decays only sub-exponentially in total compute, $\exp(-\Omega(N^{1-1/(d+1)}))$, and derives the compute-optimal branching factor and depth, together with maximum-likelihood estimators and delta-method confidence intervals for end-to-end success. Empirically, the model predicts observed accuracy to within $\sim 1\%$ across GPT-5, Claude Opus 4.6, Claude Sonnet 4.6, and Gemini 3.1 Pro; at a fixed 48-step budget a chain succeeds $\approx 0.2\%$ of the time while a depth-3 tree succeeds 97.9%.

This yields the engineering principle that organizes the rest of this portfolio.

Principle 1 (Supervised, tree-structured autonomy). Do not ship from one long autonomous chain. Decompose work into short, independently verified units; aggregate across them with explicit checkpoints (validation, evidence, human review). Keep per-step failure low and let aggregation, not chain length, carry reliability.

4 SYMPHONY: A Supervised-Autonomy Delivery System

SYMPHONY (github.com/dm3n/symphony) is the clearest demonstration of my agentic-software engineering. It is a Jira-native, evidence-gated runner that takes a product ticket from backlog to a reviewed, evidence-backed pull request, with a human as the sole approval authority. I designed and built it as CTO of Finsider, where it runs live against a multi-repository codebase. It is a ~2,300-line, standard-library Python orchestrator — no agent framework — chosen for auditability and portability.

Architecture. The system treats the coding agent as a capable but unprivileged developer: it may implement, validate, and produce evidence, but never holds approval authority. Its defining properties:

- **Jira as the state machine.** Ticket status and labels *are* the control flow; there is no external queue or database. Because state is re-derived from Jira on every poll, the daemon is crash-safe and losslessly resumable — a direct application of the “keep each step verifiable” principle.
- **Per-ticket isolation.** Each ticket runs in an isolated Git workspace on its own branch, with pinned commit identity and evidence excluded from history.
- **Acceptance contracts.** Acceptance criteria are parsed from the ticket and injected into the agent prompt as an explicit verification checklist.
- **Evidence gating.** No ticket reaches review, and no pull request is created, without PLAYWRIGHT-captured screenshots/video of the running result, integrity-checked against minimum-size thresholds to reject empty or synthetic “evidence.”
- **Intent-classified human review.** Reviewer comments are classified (question vs. rework vs. approval vs. neutral), so the loop answers questions in place and re-runs the agent only on genuine change requests.
- **Cross-repo scope critic.** The system detects when a ticket requires changes across multiple repositories and blocks review if coverage is incomplete.
- **Safety by construction.** `--force-with-lease` only; recursive process-tree cleanup and a dev-server watchdog; bounded failure auto-retry with cooldown; a single-instance file lock.

- **Two deployment targets.** macOS `launchd` for local operation and an always-on GCP Compute Engine VM under `systemd` for production.

The invariant. The controls compose to a single guarantee: SYMPHONY can never merge code, never force-clobber a branch, never advance unverified work, and never fail silently. The maximum blast radius of any malfunction is a clearly labeled, un-merged draft pull request plus a notification — exactly the property required of an autonomous system pointed at production. The full engineering record (architecture, state machine, hardening, deployment, security, and sanitized reference interfaces) is published at github.com/dm3n/symphony.

5 A Portfolio of Agentic and AI Systems

Airbank — Quality-of-Earnings engine (github.com/dm3n/airbank). An eleven-section, RAG-grounded extraction pipeline that produces analyst-grade M&A Quality-of-Earnings workbooks in which every figure is traceable to a source document. It applies Principle 1 to document AI: rather than one long extraction pass, each section is an independently verified unit with calibrated confidence, audit-grade citation (document, page, verbatim excerpt), and a policy of flagging missing data rather than fabricating it; a deterministic path bypasses retrieval for clean spreadsheets, and a formula resolver reconciles cross-section dependencies over a live SSE stream.

Airbank — mortgage underwriting (github.com/dm3n/airbank-mortgage-platform). Full-lifecycle AI origination for Canadian brokers: Canadian-specific document extraction (T4, NOA, pay-stub, LOE); *fraud-signal detection* via PDF-metadata, font-consistency, and name/income cross-checks; a GDS/TDS engine with stress-test rate; one-call intake orchestration; real-time upload verification; and an autonomous, cron-driven pipeline agent that generates role-aware priorities across a live deal pipeline.

Macintosh — a multi-agent engineering operating system (github.com/dm3n/macintosh). A reproducible, self-hosted environment that lets one engineer ship like a team: a six-stage agentic coding pipeline expressed as Claude Code skills; four coding agents (Claude Code, Codex, Gemini, OpenCode) sharing one context; a Personal Knowledge Brain (Obsidian + an LLM compiler) for persistent memory; and a Proxmox/Docker homelab with a human-gated approval model.

VisionClaw — a real-time multimodal agent (github.com/dm3n/VisionClaw).

An accessibility-first iOS agent (Swift/SwiftUI, Meta Wearables DAT SDK, Gemini Live over WebSocket) that streams video from Meta Ray-Ban glasses and proactively narrates the world for visually impaired users, with hands-free activation and resilient session recovery.

Further systems. **ROGI** (github.com/dm3n/rogi), an AI-native mortgage-intake funnel with a Canadian qualification engine (first version sold for \$7,500); **Sacred Secretion Agent** (github.com/dm3n/sacred-secretion-agent), a fully autonomous scheduling/email agent; **Nassau** (github.com/dm3n/nassau), a complete e-commerce platform built from scratch (Next.js, Sanity CMS, Stripe, Firebase); and a second research manuscript (github.com/dm3n/human-divinity).

6 Itinerary: Roles, Milestones, and Technical Objectives

Primary location of work: Toronto, Ontario, Canada (Antler Canada / OneEleven, 325 Front St W).

6.1 Co-founder & CEO — Flagpost AI Inc. (Sep 2025 – Q1 2026)

- Selected into the Antler Canada TOR8 residency (commenced Sep 2025) and passed the Antler Investment Committee.
- *Funding milestone:* raised \$220,000 at a \$2.2M valuation via a SAFE from Antler Canada Fund I, L.P.; incorporated Flagpost AI Inc. (Nov 2025).
- *Duties:* company formation, team and cap-table structure, product direction (AI-native sell-side M&A preparation), and investor relations.
- *Outcome:* stepped aside to pursue the broader M&A-intelligence thesis.

6.2 Founder — Airbank (Q1 2026)

- *Technical objectives:* design an AI-native operating system for M&A, beginning with the Quality-of-Earnings wedge; build the QoE and mortgage platforms above.
- *Development phases:* RAG ingestion → section-wise extraction with confidence flags → auditable source-linked cells → reconciliation/export;

mortgage intake → AI verification → underwriting audit → pipeline orchestration.

- *Customer acquisition*: signed LOIs, NDAs, and advisory agreements with senior professionals across private credit, M&A advisory, investment banking, and private equity.

6.3 Chief Technology Officer — Finsider (2026 – Present)

- *Compensation*: \$200,000 annual salary — materially above the norm for the role and tenure.
- *Technical objectives*: automate Quality of Earnings end-to-end; own the multi-repository platform (frontend, backend engine, AI/agent services) and the financial-correctness layer (Balance Sheet, P&L, Cash Flow, and Cash Proof tie-out with standardized export).
- *Signature deliverable*: designed and deployed SYMPHONY as the company’s agentic delivery infrastructure.
- *Forward plan (petition period)*: (i) harden and scale the QoE automation and agentic delivery pipeline; (ii) extend financial-statement correctness and export coverage; (iii) grow enterprise adoption of the automated QoE product; (iv) advance the supervised-autonomy research-to-production loop.

7 Education

Queen’s University — Computer Science. Left to build full-time after raising venture funding.

Repository Index

- **Portfolio (this overview)**: github.com/dm3n/portfolio
- **SYMPHONY**: github.com/dm3n/symphony
- **Macintosh (engineering OS)**: github.com/dm3n/macintosh
- **Research — reasoning reliability**: github.com/dm3n/uncertainty-propagation
- **Research — manuscript**: github.com/dm3n/human-divinity
- **Airbank QoE (private)**: github.com/dm3n/airbank

- **Airbank Mortgage** (private): github.com/dm3n/airbank-mortgage-platform
- **ROGI**: github.com/dm3n/rogi | github.com/dm3n/rogi-v2
- **VisionClaw**: github.com/dm3n/VisionClaw
- **Nassau**: github.com/dm3n/nassau
- **Sacred Secretion Agent**: github.com/dm3n/sacred-secretion-agent
- **Other**: github.com/dm3n/nodebase | github.com/dm3n/ratestore | github.com/dm3n/OpenCV

Prepared June 26, 2026. Curriculum vitae appended. Sensitive investor, customer, and financial specifics are maintained in a private data room available on request.